

Lecture 7: ECE C147A/C247A, Neural Networks & Deep Learning

Back propagation and optimization
October 19th, 2026

These slides are based on material from Professor Jonathan Kao, who also developed it based on other freely available sources including from Stanford and other resources, whom we acknowledge.

Announcements

- HW #3 is due Friday October 23, 11.59pm uploaded to Gradescope. **Note that even 1 minute late is considered a day late**

↘ your responsibility to make sure HW is uploaded!

- HW#1 solutions posted.

- **Please read the rules handout carefully including for late HWs etc.**

- To submit your Jupyter Notebook, print the notebook to a pdf with your solutions and plots filled in. The graders will grade your work from these notebooks. In the future, you will also have to print out code from any .py files that were modified.

- Discussion sessions held on Fridays 11am, will be recorded and live zoom link also is available.

- For midterm **CAE approved accommodation students have to contact CAE office to proctor exam** as we cannot do this ourselves. Please do this pro-actively.

letter size → · Nov 4, 2026 (Wed) → split across 3 rooms
← · 1 sheet of reference material allowed



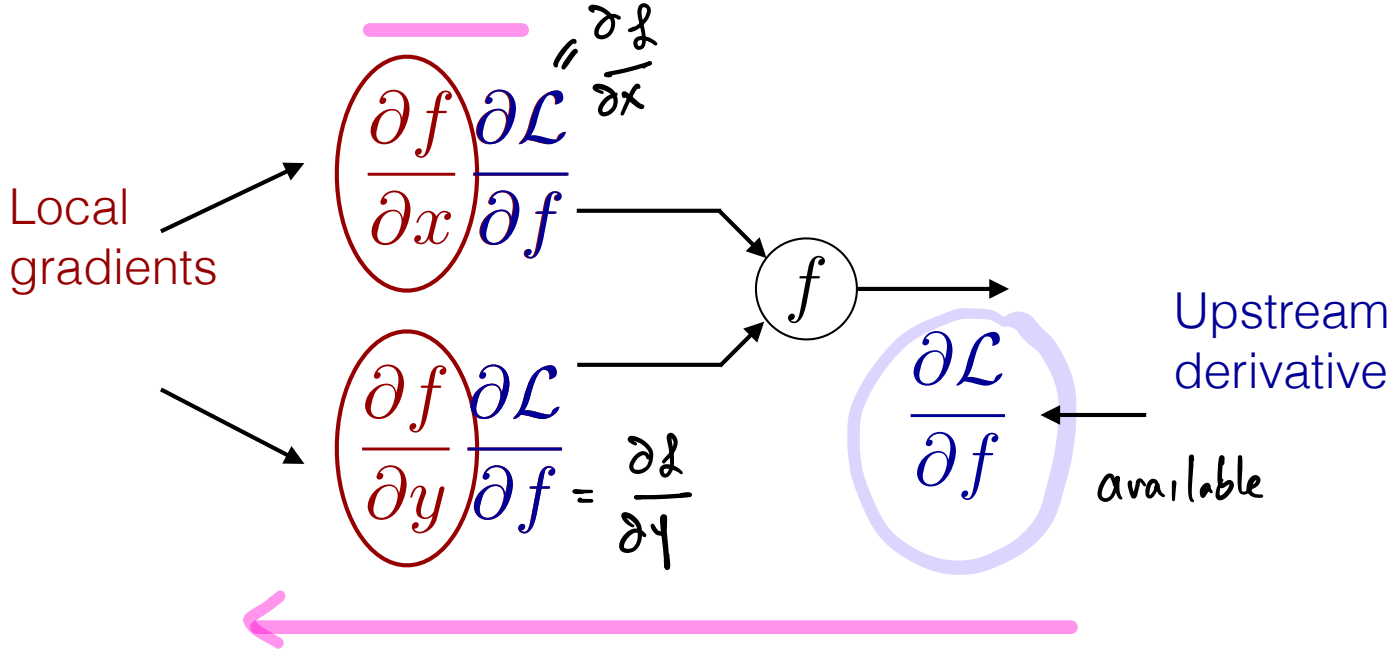
Review of previous lecture

- **Choice of non-linearity**
  - Sigmoid, hyperbolic tangent (tanh), Rectified Linear Unit (ReLU), Softplus, Leaky ReLU, and many others.
  - Matching loss function to choice of non-linearity: example through MSE and CE loss functions with sigmoid.
  - **Bottomline:** Design through lens of gradient signal and making sure it does not vanish when we need to change the parameters out of a bad solution.

- **Optimization:** Gradient descent
  - $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$, where η is the learning rate (step-size) and the gradient of the loss function $J(\boldsymbol{\theta})$.
  - Need to efficiently calculate gradient $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
  - Done through computational graph and backpropagation.

- **Backpropagation:** Basic concept is applying chain rule
  - **Forward propagation:** move forward in computation graph and calculate the values of all variables
  - **Backward propagation**: use upstream gradients combined with local gradient in the form of chain rule to find the gradient.

Idea: computational graphs apply to gradients



The basic intuition of backpropagation is that we break up the calculation of the gradient into **small and simple steps**. Each of these nodes in the graph is a straightforward gradient calculation, where we multiply an **input** (the "upstream derivative") with a **local gradient** (an application of the chain rule).

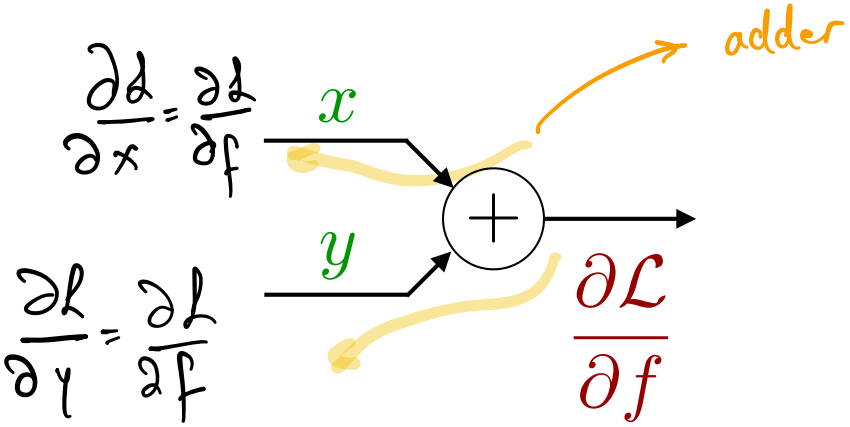
Composing all of these gradients together returns the overall gradient.

A gate view of gradients

Interpreting backpropagation as gradient “gates”:

$f = x + y$

$\frac{\partial f}{\partial x} = 1 \qquad \frac{\partial f}{\partial y} = 1$

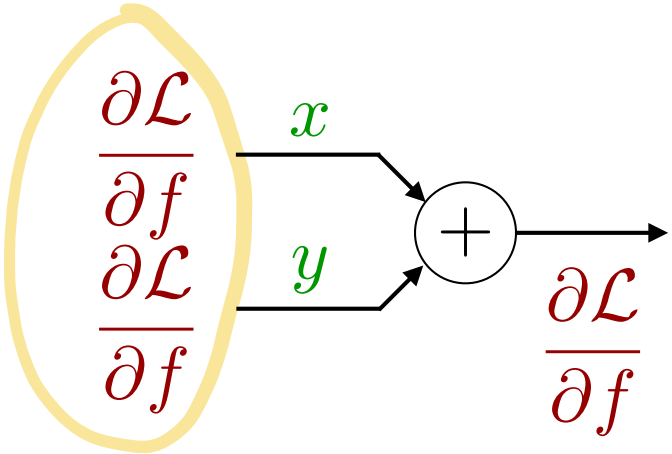




A gate view of gradients

Interpreting backpropagation as gradient “gates”:

Add gate: distributes the gradient



A gate view of gradients

Interpreting backpropagation as gradient “gates”:

Add gate: distributes the gradient

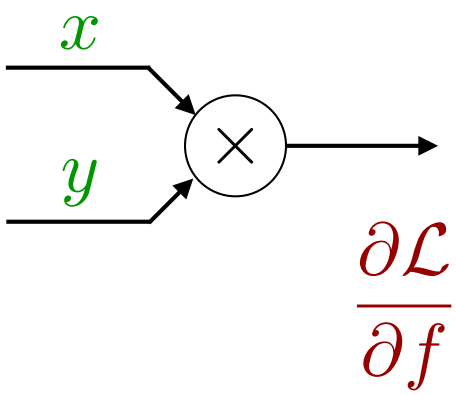
$$f = x \cdot y$$

Local gradients

$$\frac{\partial f}{\partial x} = y, \quad \frac{\partial f}{\partial y} = x$$

$$\frac{\partial L}{\partial x} = y \frac{\partial L}{\partial f}$$

$$\frac{\partial L}{\partial y} = x \frac{\partial L}{\partial f}$$

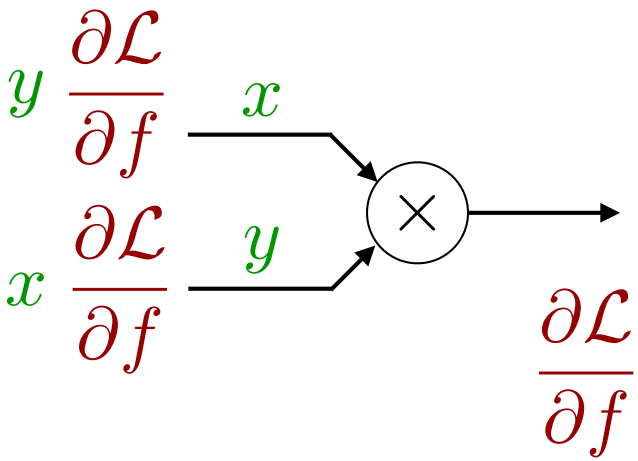




A gate view of gradients

Interpreting backpropagation as gradient "gates":

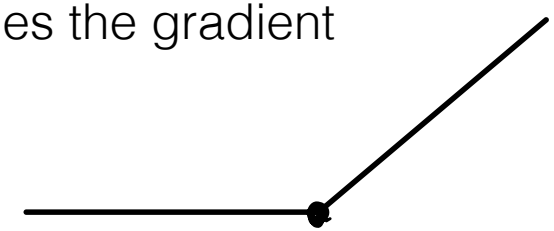
Add gate: distributes the gradient
Mult gate: switches the gradient



A gate view of gradients

Interpreting backpropagation as gradient “gates”:

Add gate: distributes the gradient
Mult gate: switches the gradient



$$f = \max(x,y) = \begin{cases} x & x > y \\ y & x \le y \end{cases}$$

$$\frac{\partial f}{\partial x} = \begin{cases} 1 & x > y \\ 0 & x \le y \end{cases}$$

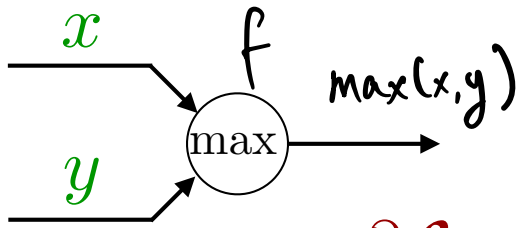
$$= \mathbb{1}_{\{x>y\}}$$

indicator function

$$\mathbb{1}_{\{A\}} = \begin{cases} 1 & A \text{ is true} \\ 0 & A \text{ is false} \end{cases}$$

condition

$$\frac{\partial L}{\partial x} = ? \; \frac{\partial L}{\partial f}$$



$$\frac{\partial \mathcal{L}}{\partial f}$$

$$= \mathbb{1}_{\{x>y\}} \frac{\partial L}{\partial f}$$

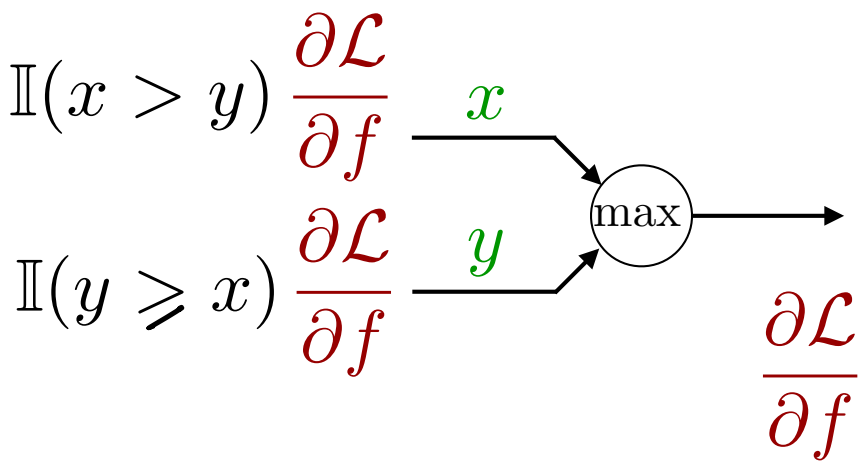
$$\frac{\partial L}{\partial y} = \mathbb{1}_{\{y \ge x\}} \frac{\partial L}{\partial f}$$



A gate view of gradients

Interpreting backpropagation as gradient “gates”:

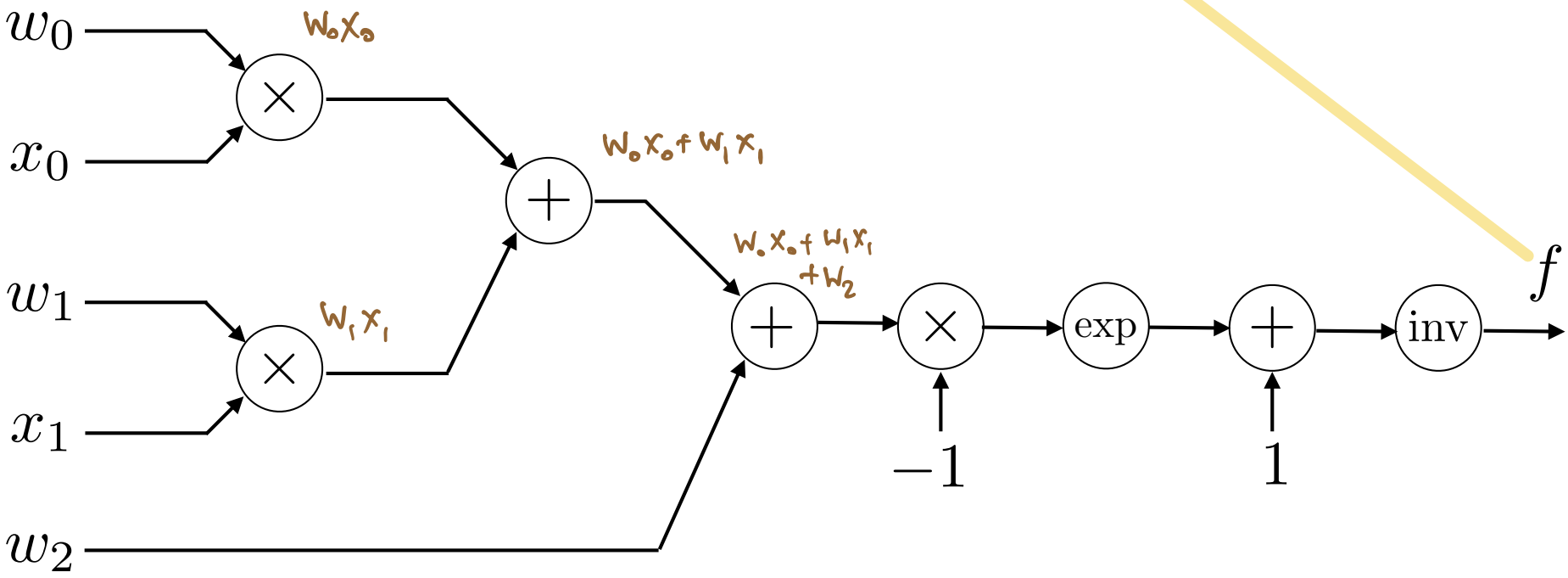
Add gate: distributes the gradient
Mult gate: switches the gradient
Max gate: routes the gradient



A more involved scalar example

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$$

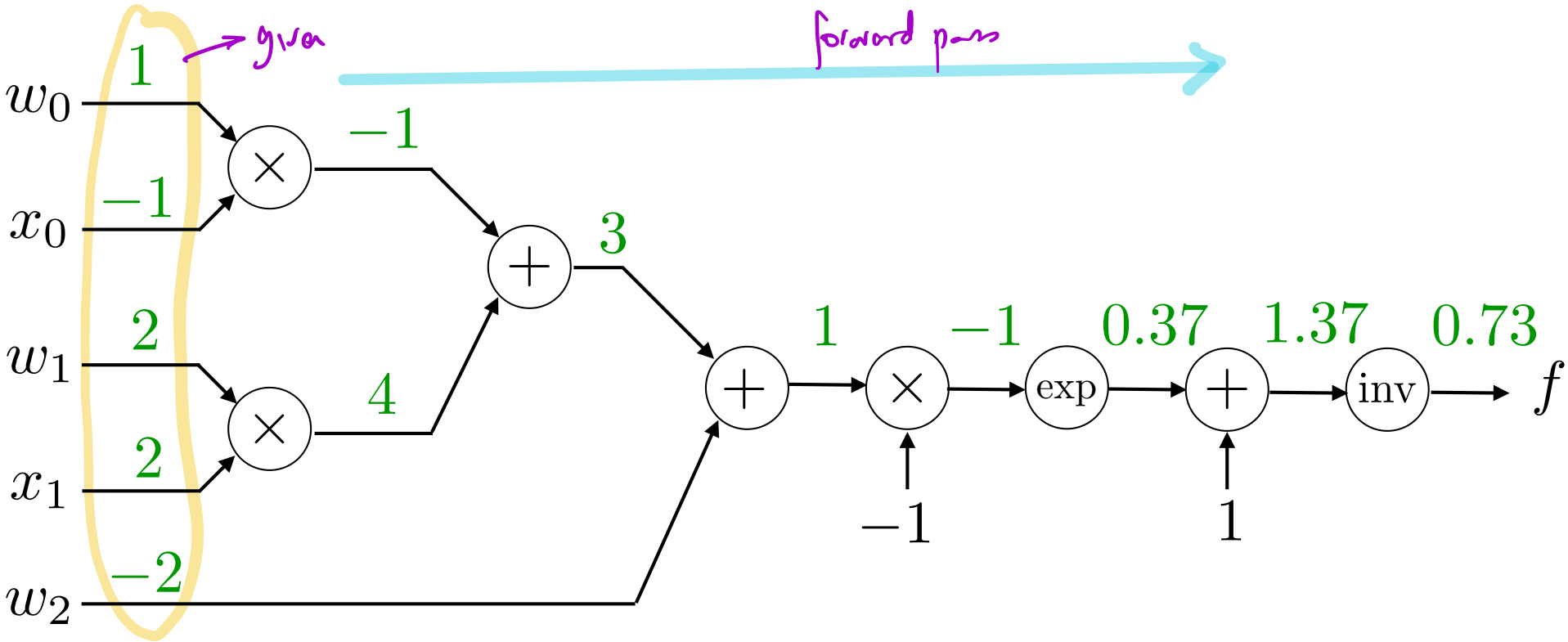
Sigmoid function



A more involved scalar example

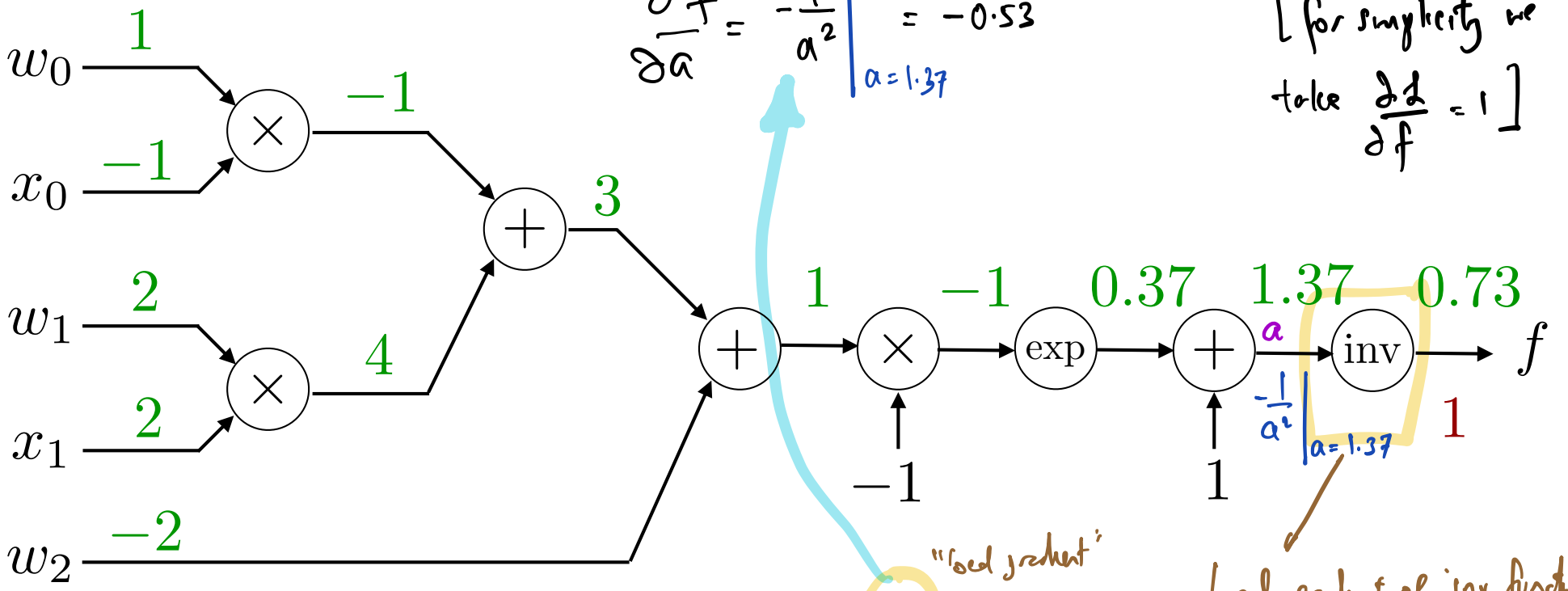
$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$$

Forward pass – Compute the values on computational graph



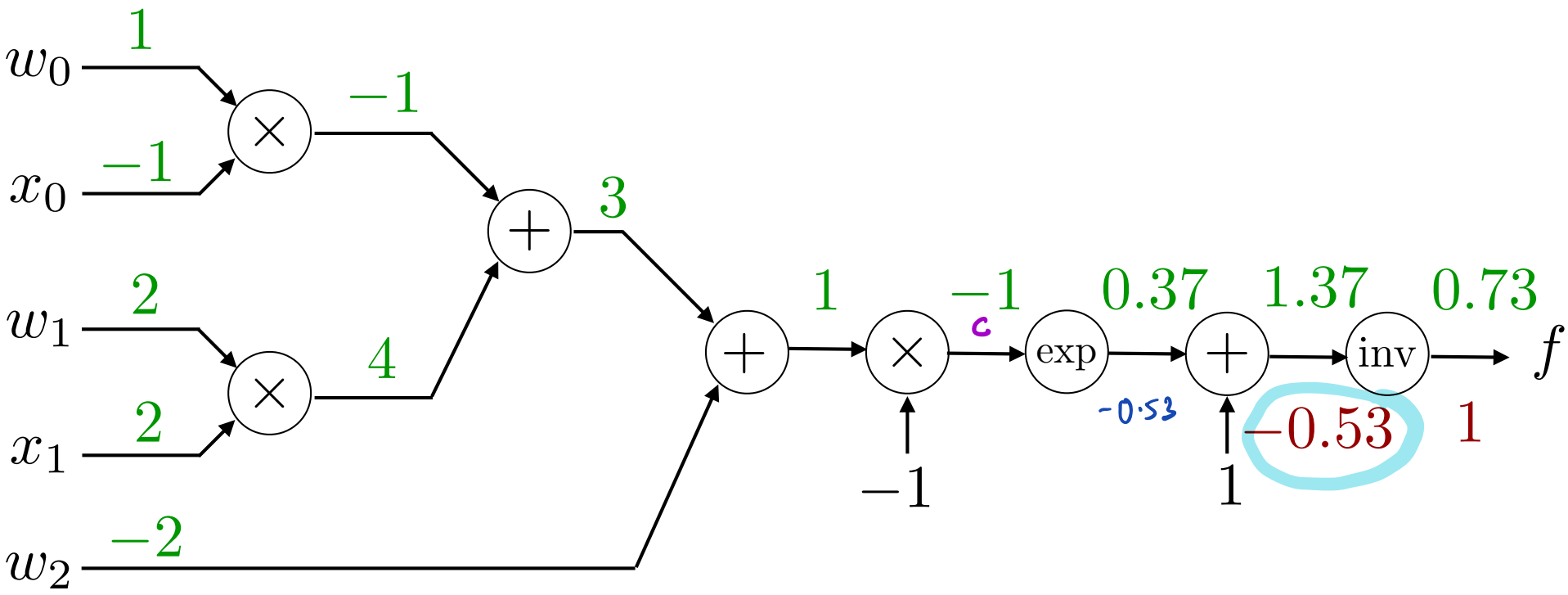
A more involved scalar example

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$$



A more involved scalar example

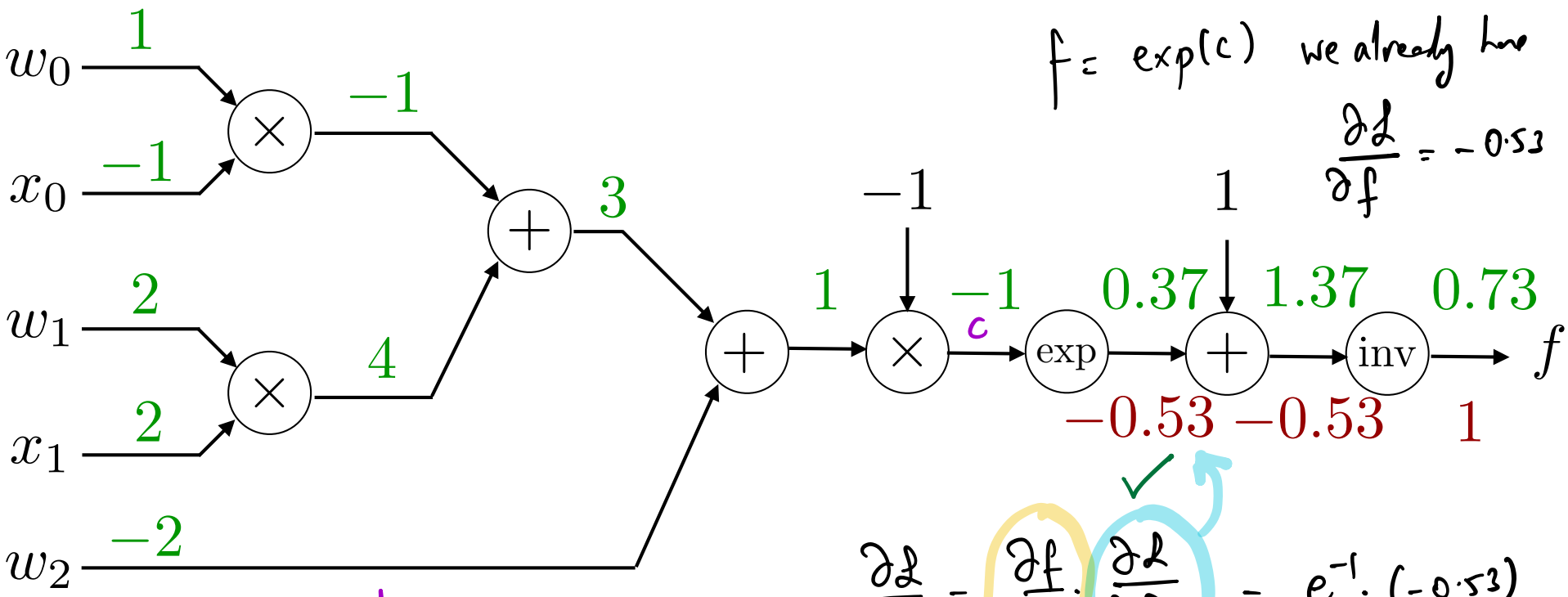
$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$$



$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial f}{\partial z}\frac{\partial \mathcal{L}}{\partial f} \qquad \frac{\partial f}{\partial z} = -\frac{1}{z^2}$$

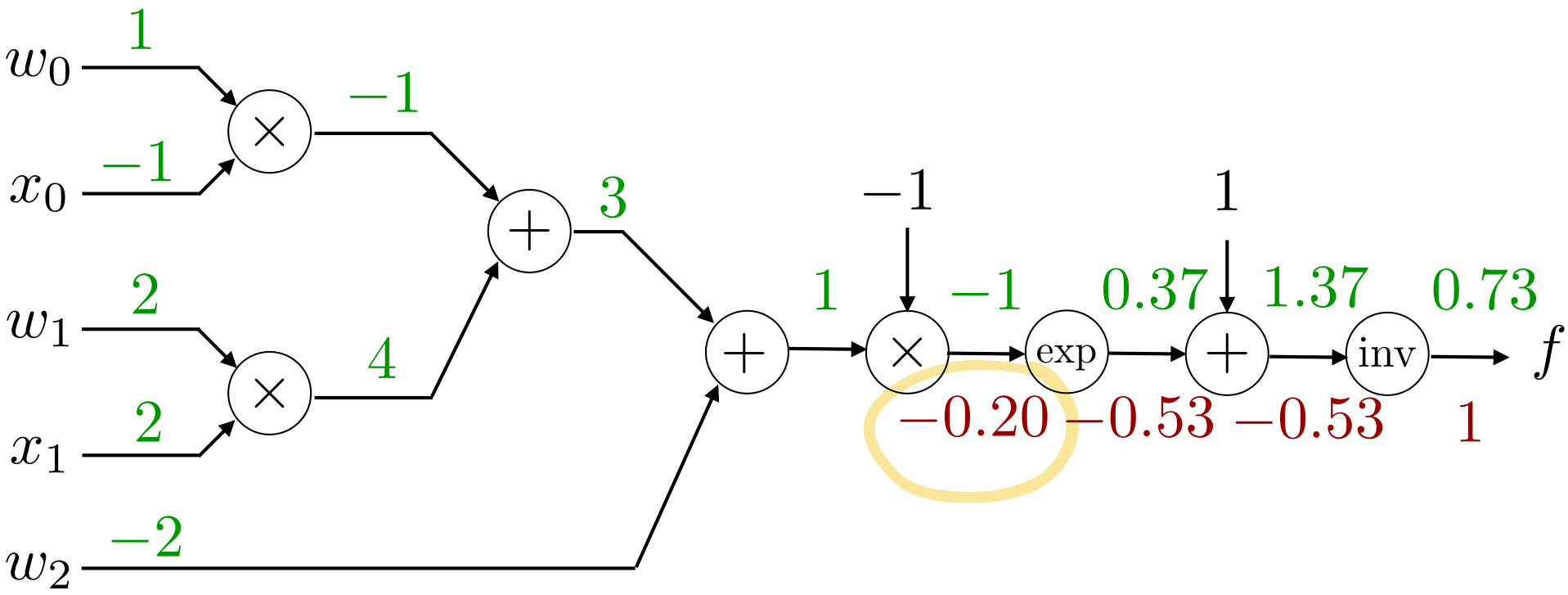
A more involved scalar example

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$$



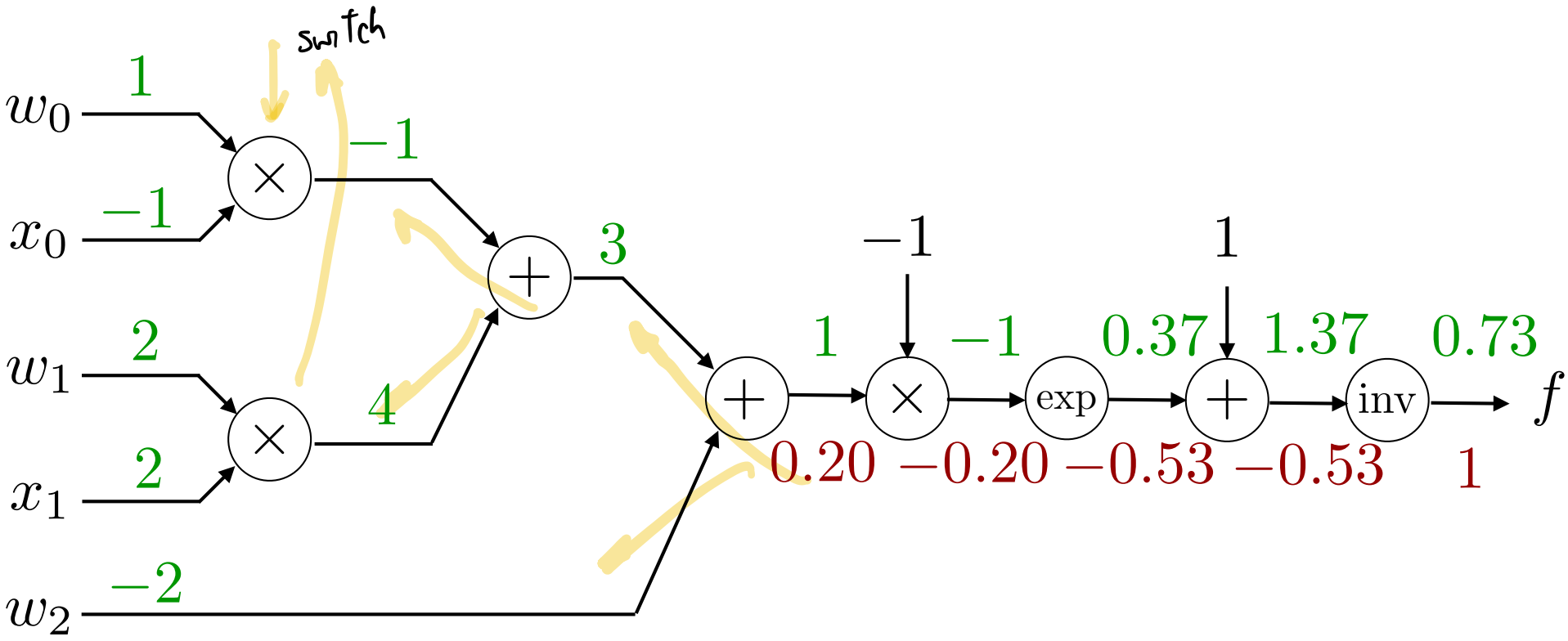
A more involved scalar example

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$$



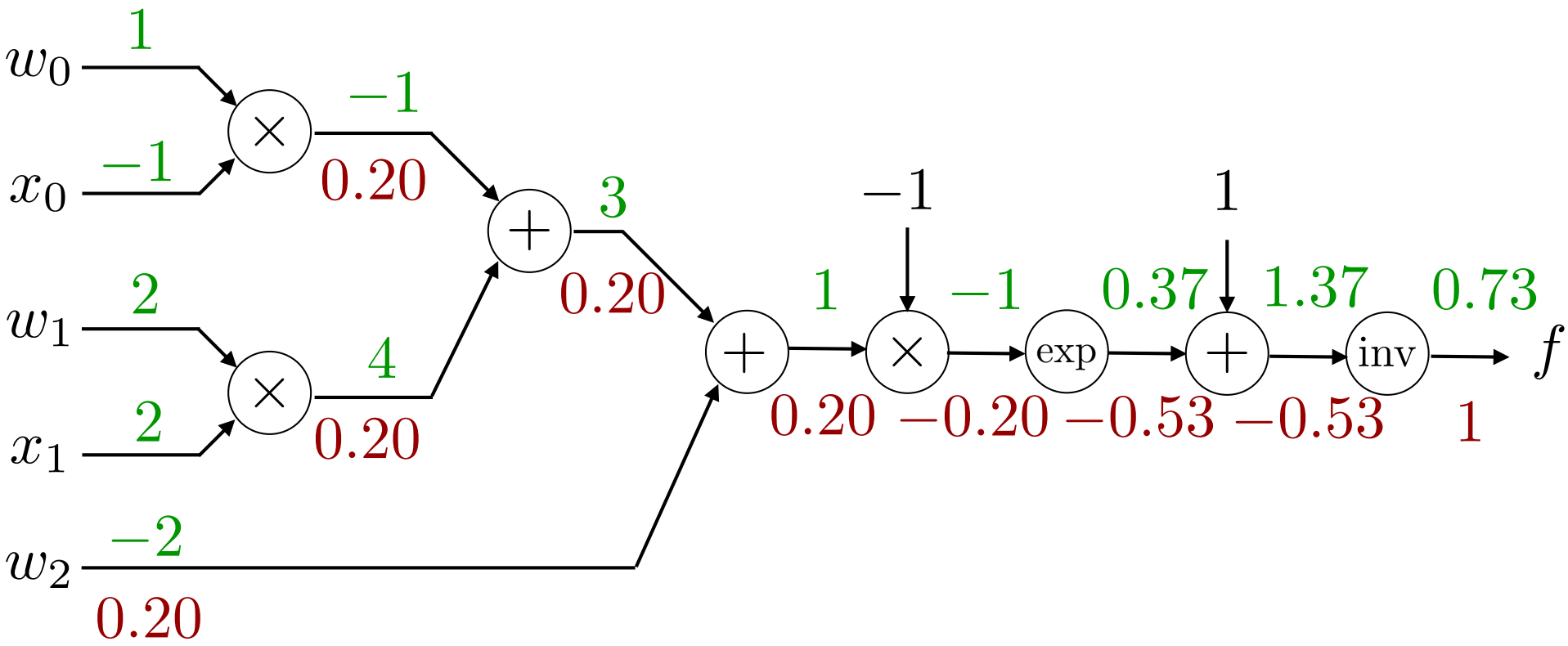
A more involved scalar example

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$$



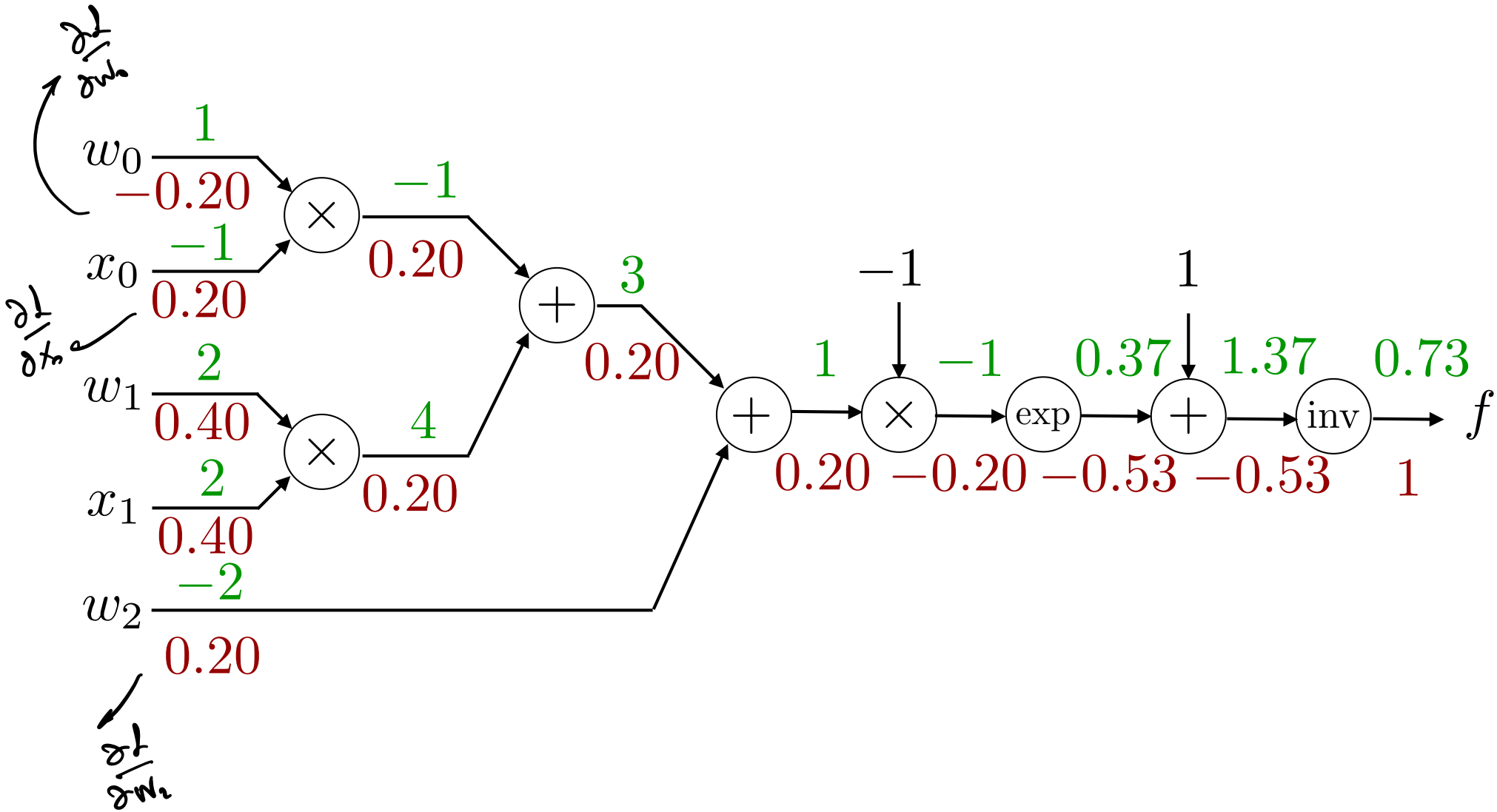
A more involved scalar example

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$$



A more involved scalar example

$$f(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$$





You can take any gradient this way

With backpropagation, as long as you can **break the computation into components where you know the local gradients, you can take the gradient of anything.**

What happens when two gradient paths converge?

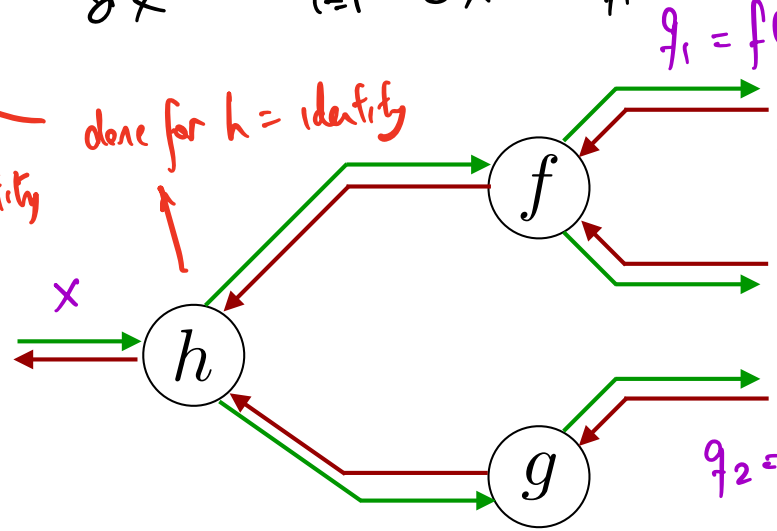
$$\frac{\partial \ell}{\partial x} = \frac{\partial \ell}{\partial q_1}\cdot\frac{\partial q_1}{\partial x} + \frac{\partial \ell}{\partial q_2}\cdot\frac{\partial q_2}{\partial x} \longrightarrow \text{law of total derivatives}$$

$$\text{In general } \frac{\partial \ell}{\partial x} = \sum_{i=1}^{n} \frac{\partial q_i}{\partial x}\cdot\frac{\partial \ell}{\partial q_i}$$

do the exercise for yourself if h is not identity

GOAL: identity

done for h = identity



Find $\frac{\partial \ell}{\partial x}$

$$\frac{\partial f}{\partial x}\frac{\partial \ell}{\partial f} + \frac{\partial g}{\partial x}\cdot\frac{\partial \ell}{\partial g}$$

if $f = g$ = identity function

then $\frac{\partial \ell}{\partial x} = \frac{\partial \ell}{\partial q_1} + \frac{\partial \ell}{\partial q_2}$

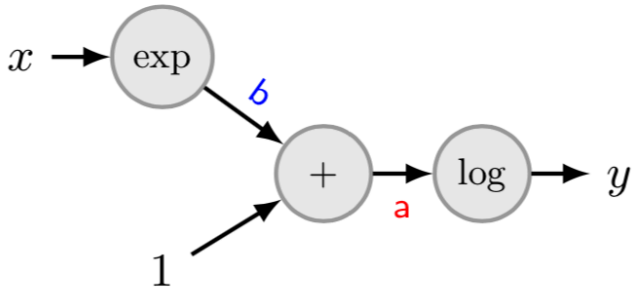
Backpropagation example: softplus

- $y = \text{softplus}(x) = \log(1 + \exp(x))$. We can analytically calculate the derivative.

$$\frac{dy}{dx} = \frac{\exp(x)}{1 + \exp(x)}$$

Let's now calculate it via backpropagation.

You can go through or do it yourself.

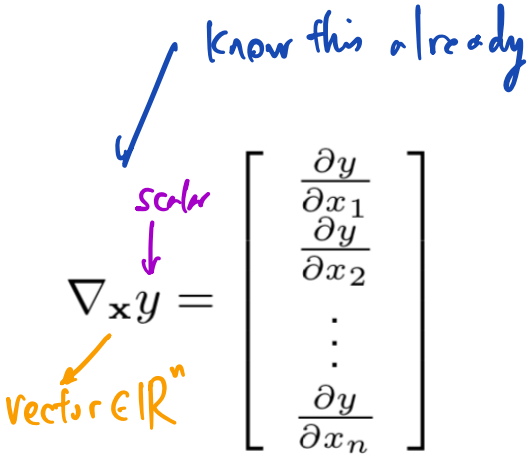


Here, we have that $b = \exp(x)$ and that $a = 1 + \exp(x)$. Applying the chain rule, we have that:

$$\begin{aligned}
\frac{dy}{da} &= \frac{d}{da}\log a = \frac{1}{a} \\
\frac{dy}{db} &= \frac{da}{db}\frac{dy}{da} = \frac{dy}{da} \\
\frac{dy}{dx} &= \frac{db}{dx}\frac{dy}{db} = \exp(x)\frac{1}{a}
\end{aligned}$$

Derivative of a scalar w.r.t. a vector

To do multivariate backpropagation, we need a multivariate chain rule.



In other words, the gradient is:

- A *vector* that is the same size as $\mathbf{x}$, i.e., if $\mathbf{x} \in \mathbb{R}^n$ then $\nabla_{\mathbf{x}} y \in \mathbb{R}^n$.
- Each dimension of $\nabla_{\mathbf{x}} y$ tells us how small changes in $\mathbf{x}$ in that dimension affect y. i.e., changing the ith dimension of $\mathbf{x}$ by a small amount, Δx_i, will change y by

$$\frac{\partial y}{\partial x_i} \Delta x_i$$

We may also denote this as:

$$(\nabla_{\mathbf{x}} y)_i \, \Delta x_i$$

Derivative of a scalar w.r.t. a matrix

The derivative of a scalar, y, with respect to a matrix, $\mathbf{A} \in \mathbb{R}^{m \times n}$, is given by:

$$\nabla_{\mathbf{A}} y = \begin{bmatrix} \frac{\partial y}{\partial a_{11}} & \frac{\partial y}{\partial a_{12}} & \cdots & \frac{\partial y}{\partial a_{1n}} \\ \frac{\partial y}{\partial a_{21}} & \frac{\partial y}{\partial a_{22}} & \cdots & \frac{\partial y}{\partial a_{2n}} \\ \cdots & \cdots & \ddots & \vdots \\ \frac{\partial y}{\partial a_{m1}} & \frac{\partial y}{\partial a_{m2}} & \cdots & \frac{\partial y}{\partial a_{mn}} \end{bmatrix}$$

Like the gradient, the i, jth element of $\nabla_{\mathbf{A}} y$ tells us how small changes in a_{ij} affect y.

Note:

- If you search for the derivative of a scalar with respect to a matrix, you may find people give a transposed definition to the one above.
- Both are valid, but you must be consistent with your notation and use the correct rules. Our notation is called "denominator layout" notation; the other layout is called "numerator layout" notation.
- In the denominator layout, the dimensions of $\nabla_{\mathbf{A}} y$ and $\mathbf{A}$ are the same. The same holds for the gradient, i.e., the dimensions of $\nabla_{\mathbf{x}} y$ and $\mathbf{x}$ are the same. In the numerator layout notation, the dimensions are transposed.
- More on this later, but in "denominator layout," the chain rule goes right to left as opposed to left to right.

$\underline{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$

Derivative of a vector w.r.t. a vector

$\Delta \underline{x} = \begin{bmatrix} \Delta x_1 \\ \vdots \\ \Delta x_m \end{bmatrix}$

Let $\mathbf{y} \in \mathbb{R}^n$ be a function of $\mathbf{x} \in \mathbb{R}^m$. What dimensionality should the derivative of $\mathbf{y}$ with respect to $\mathbf{x}$ be?

- e.g., to see how $\Delta\mathbf{x}$ modifies y_i, we would calculate:

$$\Delta y_i = \nabla_{\mathbf{x}} y_i \cdot \Delta \mathbf{x}$$

Remember if scalar

$\Delta y \approx \frac{\partial y}{\partial x} \Delta x$

$\Delta \underline{x} \in \mathbb{R}^m$ small "wiggle" in $\underline{x}$

- This suggests that the derivative ought to be an $n \times m$ matrix, denoted $\mathbf{J}$, of the form:

Question: how do we do this if $\underline{x}$ is vector

$\Delta y_i \approx \sum_{j=1}^{m} \frac{\partial y_i}{\partial x_j} \Delta x_j$

$\approx (\nabla_{\underline{x}} y_i)^T \Delta \underline{x}$

$$\mathbf{J} = \begin{bmatrix} (\nabla_{\mathbf{x}} y_1)^T \\ (\nabla_{\mathbf{x}} y_2)^T \\ \vdots \\ (\nabla_{\mathbf{x}} y_n)^T \end{bmatrix} \quad \Delta \underline{y} \approx \begin{bmatrix} (\nabla_{\underline{x}} y_1)^T \\ \vdots \\ (\nabla_{\underline{x}} y_n)^T \end{bmatrix} \Delta \underline{x}$$

$$= \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \cdots & \frac{\partial y_1}{\partial x_m} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_2}{\partial x_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial x_1} & \frac{\partial y_n}{\partial x_2} & \cdots & \frac{\partial y_n}{\partial x_m} \end{bmatrix} = \text{Jacobian}$$

denominator layout

The matrix would tell us how a small change in $\Delta\mathbf{x}$ results in a small change in $\Delta\mathbf{y}$ according to the formula:

$$\Delta \mathbf{y} \approx \mathbf{J} \Delta \mathbf{x}$$

Derivative of a vector w.r.t. a vector (cont.)

The matrix $\mathbf{J}$ is called the Jacobian matrix.

A word on notation:

- In the denominator layout definition, the denominator vector changes in a column.

$\nabla_{\underline{x}} \underline{y}$ is $m \times n$ matrix

$$\nabla_{\mathbf{x}}\mathbf{y} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_2}{\partial x_1} & \cdots & \frac{\partial y_n}{\partial x_1} \\ \frac{\partial y_1}{\partial x_2} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_n}{\partial x_2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_m} & \frac{\partial y_2}{\partial x_m} & \cdots & \frac{\partial y_n}{\partial x_m} \end{bmatrix} \leftarrow \text{denominator layout}$$

$\mathbf{x} \in \mathbb{R}^n$, $\mathbf{y} \in \mathbb{R}^m$

$$\triangleq \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \underline{\underline{J}}^T$$

- Hence the notation we use for the Jacobian would be:

$$\mathbf{J} = (\nabla_{\mathbf{x}}\mathbf{y})^T = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^T$$

$\Delta \underline{y} \approx \underline{\underline{J}} \Delta \underline{x} = (\nabla_{\underline{x}} \underline{y})^T \Delta \underline{x}$

Derivative of a vector w.r.t. a vector

Example: derivative of a vector with respect to a vector

ALERT notation dimension change

The following derivative will appear later on in the class. Let $\mathbf{W} \in \mathbb{R}^{h \times n}$ and $\mathbf{x} \in \mathbb{R}^n$. We would like to calculate the derivative of $f(\mathbf{x}) = \mathbf{W}\mathbf{x}$ with respect to $\mathbf{x}$.

$\underline{y} = f(\underline{x}) = \underline{W}\,\underline{x}$

$$\nabla_{\mathbf{x}} \mathbf{W}\mathbf{x} = \nabla_{\mathbf{x}} \begin{bmatrix} w_{11}x_1 + w_{12}x_2 + \cdots + w_{1n}x_n \\ w_{21}x_1 + w_{22}x_2 + \cdots + w_{2n}x_n \\ \vdots \\ w_{h1}x_1 + w_{h2}x_2 + \cdots + w_{hn}x_n \end{bmatrix} \rightarrow \underline{W}\,\underline{x}$$

$$\underline{W} = \begin{bmatrix} w_{11} & \cdots & w_{1n} \\ \vdots & & \\ w_{m1} & \cdots & w_{mn} \end{bmatrix} \qquad \underline{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

$$= \begin{bmatrix} w_{11} & w_{21} & \cdots & w_{h1} \\ w_{12} & w_{22} & \cdots & w_{h2} \\ \vdots & \vdots & \ddots & \vdots \\ w_{1n} & w_{2n} & \cdots & w_{hn} \end{bmatrix} \Rightarrow \nabla_{\underline{x}} [\underline{W}\underline{x}] = \underline{W}^T$$

$$= \mathbf{W}^T$$

$\frac{\partial y_1}{\partial \underline{x}}$ $\quad \frac{\partial y_1}{\partial x_1}$ $\quad \partial y_1 / \partial x_n$

Hessian: the generalization of the second derivative

$f(\underline{x}) \in \mathbb{R}$ $\quad \underline{x} \in \mathbb{R}^m$ $\quad \nabla_{\underline{x}} f(\underline{x}) \in \mathbb{R}^m$ $\quad \nabla_{\underline{x}} [\nabla_{\underline{x}} f(\underline{x})]$

↘ "2nd derivative"

Hessian

The Hessian matrix of a function $f(\mathbf{x})$ is a square matrix of second-order partial derivatives of $f(\mathbf{x})$. It is composed of elements:

$$\mathbf{H} = \begin{bmatrix} \frac{\partial f}{\partial x_1^2} & \frac{\partial f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial f}{\partial x_1 \partial x_n} \\ \frac{\partial f}{\partial x_2 x_1} & \frac{\partial f}{\partial x_2^2} & \cdots & \frac{\partial f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial x_m x_1} & \frac{\partial f}{\partial x_m \partial x_2} & \cdots & \frac{\partial f}{\partial x_m^2} \end{bmatrix}$$

multivariate generalization of the 2nd derivative

We can denote this matrix as $\nabla_{\mathbf{x}} (\nabla_{\mathbf{x}} f(\mathbf{x}))$. We often denote this simply as $\nabla_{\mathbf{x}}^2 f(\mathbf{x})$.

$\frac{\partial^2 f}{\partial x^2}$ in scalar case

we used this extensively in backpropagation

$$\frac{\partial L}{\partial x} = \frac{\partial f}{\partial x} \cdot \frac{\partial L}{\partial f}$$

"local" "upstream"

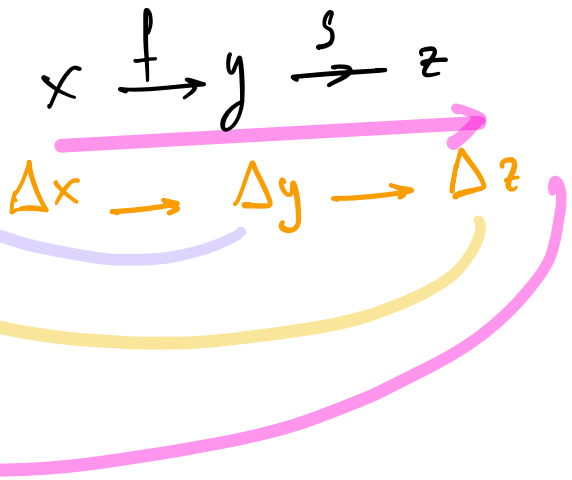
The scalar chain rule

The scalar chain rule states that if $y = f(x)$ and $z = g(y)$, then

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

Intuitively: the chain rule tells us that a small change in x will cause a small change in y that will in turn cause a small change in z, i.e., for apropriately small Δx,

$$\begin{aligned} \Delta y &\approx \frac{dy}{dx}\Delta x \\ \Delta z &\approx \frac{dz}{dy}\Delta y \\ &= \frac{dz}{dy}\frac{dy}{dx}\Delta x \end{aligned}$$



GOAL: To extend this to vectors & matrices

MOTIVATION We want to do BP through NN which have vectors & matrices.

Vector chain rule

$\underline{y} = f(\underline{x})$, $\underline{z} = g(\underline{y}) \Rightarrow \underline{z} = g(f(\underline{x}))$ $\quad \underline{x} \in \mathbb{R}^m$, $\underline{y} \in \mathbb{R}^n$, $\underline{z} \in \mathbb{R}^p$

1. $\Delta \underline{x} \longrightarrow \Delta \underline{z}$

$$\Delta \underline{z} \approx \left(\frac{\partial \underline{z}}{\partial \underline{x}}\right)^T \Delta \underline{x} \qquad (*1)$$

Jacobian

2. $\Delta \underline{x} \longrightarrow \Delta \underline{y} \longrightarrow \Delta \underline{z}$

$$\approx \left(\frac{\partial \underline{y}}{\partial \underline{x}}\right)^T \Delta \underline{x} \qquad \left(\frac{\partial \underline{z}}{\partial \underline{y}}\right)^T \Delta \underline{y}$$

$$\left(\frac{\partial \underline{z}}{\partial \underline{y}}\right)^T \left[\left(\frac{\partial \underline{y}}{\partial \underline{x}}\right)^T \Delta \underline{x}\right]$$

$$\Delta \underline{z} \approx \left(\frac{\partial \underline{z}}{\partial \underline{y}}\right)^T \left(\frac{\partial \underline{y}}{\partial \underline{x}}\right)^T \Delta \underline{x} \qquad (*2)$$

$(*1) = (*2)$

$$\Rightarrow \left(\frac{\partial \underline{z}}{\partial \underline{x}}\right)^T = \left(\frac{\partial \underline{z}}{\partial \underline{y}}\right)^T \left(\frac{\partial \underline{y}}{\partial \underline{x}}\right)^T$$

$$\Rightarrow \left(\frac{\partial \underline{z}}{\partial \underline{x}}\right) = \left(\frac{\partial \underline{y}}{\partial \underline{x}}\right) \left(\frac{\partial \underline{z}}{\partial \underline{y}}\right) \qquad n \times p$$

$m \times p$ $\quad m \times n$

ORDER MATTERS



Vector chain rule

Chain rule for vector valued functions (cont.)

- Intuitively, a small change $\Delta\mathbf{x}$ affects $\Delta\mathbf{z}$ through the Jacobian $(\nabla_{\mathbf{x}}\mathbf{z})^T$

$$\Delta\mathbf{z} \approx (\nabla_{\mathbf{x}}\mathbf{z})^T \Delta\mathbf{x} \tag{1}$$

- The chain rule is intuitive, since:

$$\begin{aligned} \Delta\mathbf{y} &\approx (\nabla_{\mathbf{x}}\mathbf{y})^T \Delta\mathbf{x} \\ \Delta\mathbf{z} &\approx (\nabla_{\mathbf{y}}\mathbf{z})^T \Delta\mathbf{y} \end{aligned}$$

What we just did

Composing these, we have that:

$$\Delta\mathbf{z} \approx (\nabla_{\mathbf{y}}\mathbf{z})^T (\nabla_{\mathbf{x}}\mathbf{y})^T \Delta\mathbf{x} \tag{2}$$

- Combining equations (1) and (2), we arrive at:

$$(\nabla_{\mathbf{x}}\mathbf{z})^T = (\nabla_{\mathbf{y}}\mathbf{z})^T (\nabla_{\mathbf{x}}\mathbf{y})^T$$

which, after transposing both sides, reduces to the (right to left) chain rule:

$$\nabla_{\mathbf{x}}\mathbf{z} = \nabla_{\mathbf{x}}\mathbf{y}\, \nabla_{\mathbf{y}}\mathbf{z}$$



Chain rule for vector valued functions

In the "denominator" layout, the chain rule runs from right to left. We won't derive this, but we will check the dimensionality and intuition.

Let $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$, and $\mathbf{z} \in \mathbb{R}^p$. Further, let $\mathbf{y} = f(\mathbf{x})$ for $f : \mathbb{R}^m \to \mathbb{R}^n$ and $\mathbf{z} = g(\mathbf{y})$ for $g : \mathbb{R}^n \to \mathbb{R}^p$. Then,

$$\nabla_{\mathbf{x}} \mathbf{z} \quad = \quad \nabla_{\mathbf{x}} \mathbf{y} \, \nabla_{\mathbf{y}} \mathbf{z}$$

Equivalently:

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} \quad = \quad \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial \mathbf{z}}{\partial \mathbf{y}}$$

- Note that $\nabla_{\mathbf{x}} \mathbf{z}$ should have dimensionality $\mathbb{R}^{m \times p}$.
- As $\nabla_{\mathbf{x}} \mathbf{y} \in \mathbb{R}^{m \times n}$ and $\nabla_{\mathbf{y}} \mathbf{z} \in \mathbb{R}^{n \times p}$, the operations are dimension consistent.

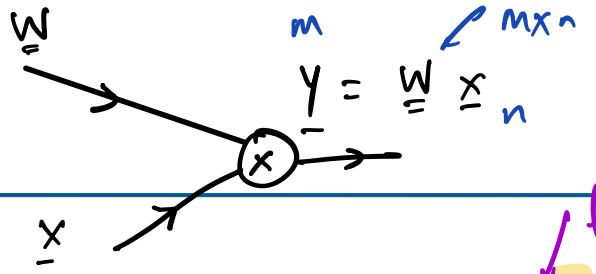
Derivatives of tensors

Occasionally in this class, we may need to take a derivative that is more than 2-dimensional. For example, we may want to take the derivative of a vector with respect to a matrix. This would be a 3-dimensional tensor. The definition for this would be as you expect. In particular, if $\mathbf{z} \in \mathbb{R}^p$ and $\mathbf{W} \in \mathbb{R}^{m \times n}$, then $\nabla_{\mathbf{W}} \mathbf{z}$ is a three-dimensional tensor with shape $\mathbb{R}^{m \times n \times p}$. Each $m \times n$ slice (of which there are p) is the matrix derivative $\nabla_{\mathbf{W}} z_i$.

Note, these are sometimes a headache to work with. We typically can find a shortcut to perform operations without having to compute and store these high-dimensional tensor derivatives. The next slides show an example.

Back to backpropagation

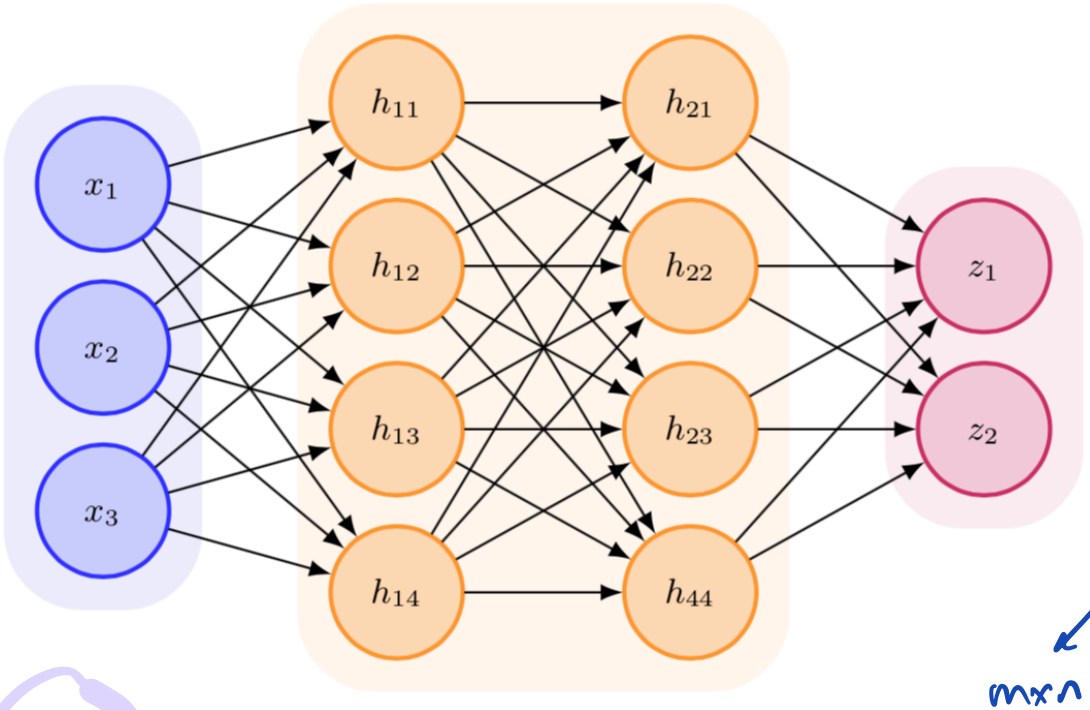
$\mathcal{L}$ → loss function scalar.



$$\frac{\partial y}{\partial x} = W^T \Rightarrow \frac{\partial \mathcal{L}}{\partial x} = W^T \frac{\partial \mathcal{L}}{\partial y}$$

local

upstream



To do $\frac{\partial \mathcal{L}}{\partial W}$ is slightly more involved

Can also do for matrices

$$Y = W X$$

$n \times m$, $m \times p$, $n \times p$

$$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial Y}{\partial X} \cdot \frac{\partial \mathcal{L}}{\partial Y}$$

local; upstream known

$(k,l)^{th}$ elem $\sum_{i=1}^{n} \sum_{j=1}^{p} \left(\nabla_X Y\right)_{k,l,i,j} \left(\nabla_Y \mathcal{L}\right)_{ij}$

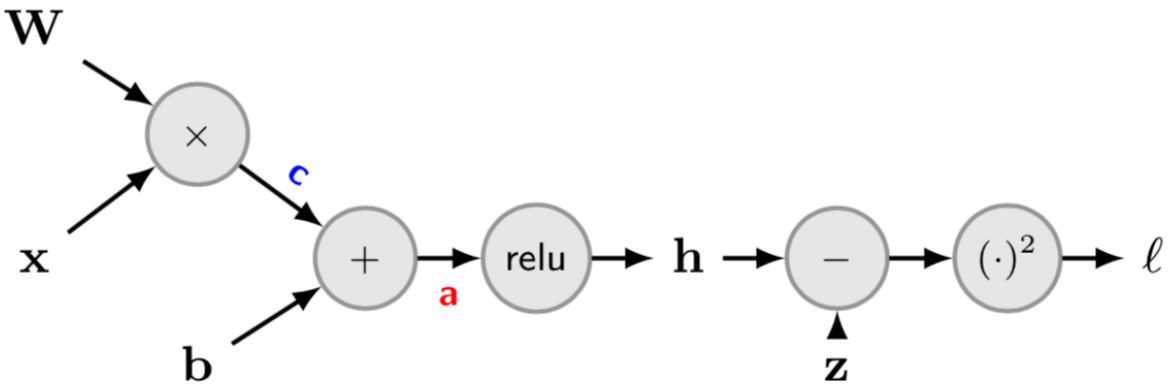
$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial Y} X^T$$

$m \times 1$, $1 \times n$, $m \times n$

will explain soon for $Y = W X$

Backpropagation: neural network layer

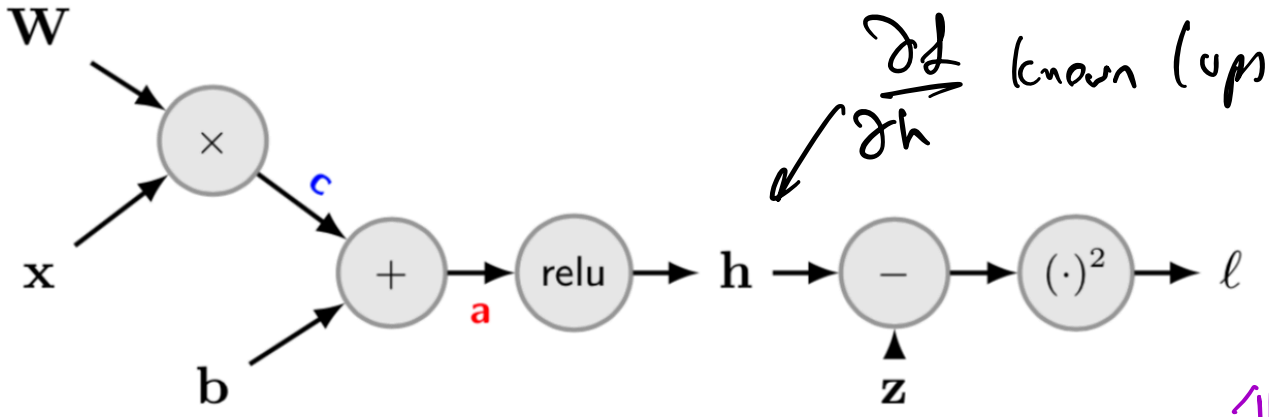
Here, $\mathbf{h} = \mathrm{ReLU}(\mathbf{W}\mathbf{x} + \mathbf{b})$, with $\mathbf{h} \in \mathbb{R}^h$, $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{W} \in \mathbb{R}^{h \times m}$, and $\mathbf{b} \in \mathbb{R}^h$. While many output cost functions might be used, let's consider a simple squared-loss output $\ell = (\mathbf{h} - \mathbf{z})^2$ where $\mathbf{z}$ is some target value.



A few things to note:

- Note that $\nabla_{\mathbf{h}} \ell = 2(\mathbf{h} - \mathbf{z})$. We will start backpropagation at $\mathbf{h}$ rather than at ℓ.
- In the following backpropagation, we'll have need for elementwise multiplication. This is formally called a Hadamard product, and we will denote it via $\odot$. Concretely, the ith entry of $\mathbf{x} \odot \mathbf{y}$ is given by $x_i y_i$.

Backpropagation for a neural network layer



GOAL: Backpropagate $\frac{\partial \ell}{\partial h}$ through this

$$\underline{h} = \text{Relu}(\underline{a}) = \begin{bmatrix} \text{relu}(a_1) \\ \vdots \\ \text{relu}(a_n) \end{bmatrix}$$

$$\text{relu}(a_i) = \max(a_i, 0) = \begin{cases} a_i & \text{if } a_i \geq 0 \\ 0 & \text{else} \end{cases}$$

$$\mathbb{1}_{\{a_i \geq 0\}} = \begin{cases} 1 & a_i \geq 0 \\ 0 & \text{else} \end{cases}$$

$$\frac{\partial \underline{h}}{\partial \underline{a}} = \begin{bmatrix} \mathbb{1}_{\{a_1 \geq 0\}} \\ \vdots \\ \mathbb{1}_{\{a_n \geq 0\}} \end{bmatrix} = \mathbb{1}_{\{\underline{a} \geq 0\}} \quad \rightarrow \text{short hand}$$

$$\frac{\partial \ell}{\partial \underline{a}} = \frac{\partial \underline{h}}{\partial \underline{a}} \cdot \frac{\partial \ell}{\partial \underline{h}}$$

$$\frac{\partial \ell}{\partial \underline{a}} = \mathbb{1}_{\{\underline{a} \geq 0\}} \frac{\partial \ell}{\partial \underline{h}}$$

$$\frac{\partial \ell}{\partial c} = \frac{\partial \ell}{\partial b} = \frac{\partial \ell}{\partial a}, \quad \frac{\partial \ell}{\partial \underline{W}} = \frac{\partial \ell}{\partial c} \cdot \underline{x}^T$$

$\rightarrow$ stated before

Backpropagation: neural network layer (cont.)

Applying the chain rule, we have:

$$\begin{aligned}
\frac{\partial \ell}{\partial \mathbf{a}} &= \mathbb{I}(\mathbf{a} > 0) \odot \frac{\partial \ell}{\partial \mathbf{h}} \\
\frac{\partial \ell}{\partial \mathbf{c}} &= \frac{\partial \ell}{\partial \mathbf{a}} \\
\frac{\partial \ell}{\partial \mathbf{x}} &= \frac{\partial \mathbf{c}}{\partial \mathbf{x}} \frac{\partial \ell}{\partial \mathbf{c}} \\
&= \mathbf{W}^T \frac{\partial \ell}{\partial \mathbf{c}}
\end{aligned}$$

A few notes:

- For $\frac{\partial \mathbf{c}}{\partial \mathbf{x}}$, see example in the Tools notes.
- Why was the chain rule written right to left instead of left to right? This turns out to be a result of our convention of how we defined derivatives (using the "denominator layout notation") in the linear algebra notes.
- Though maybe not the most satisfying answer, you can always check the order of operations is correct by considering non-square matrices, where the dimensionality must be correct.



Now we have the gradients, we can do gradient descent

With the gradients of the parameters, we can go ahead and now apply our learning algorithm, gradient descent.

However, we'll soon find that if we do this naively, performance won't be great **for neural networks** (you'll see this on HW).

There are many other important considerations we now need to consider before we can train these networks adequately.

Derivatives of tensors

Occasionally in this class, we may need to take a derivative that is more than 2-dimensional. For example, we may want to take the derivative of a vector with respect to a matrix. This would be a 3-dimensional tensor. The definition for this would be as you expect. In particular, if $\mathbf{z} \in \mathbb{R}^p$ and $\mathbf{W} \in \mathbb{R}^{m \times n}$, then $\nabla_{\mathbf{W}} \mathbf{z}$ is a three-dimensional tensor with shape $\mathbb{R}^{m \times n \times p}$. Each $m \times n$ slice (of which there are p) is the matrix derivative $\nabla_{\mathbf{W}} z_i$.

Note, these are sometimes a headache to work with. We typically can find a shortcut to perform operations without having to compute and store these high-dimensional tensor derivatives. The next slides show an example.

Tensor derivatives

NOT TESTED

$\underline{y} = \underline{\underline{W}}\,\underline{x} \qquad \underline{y} \in \mathbb{R}^m, \ \underline{x} \in \mathbb{R}^n, \ \underline{\underline{W}} \in \mathbb{R}^{m \times n}$

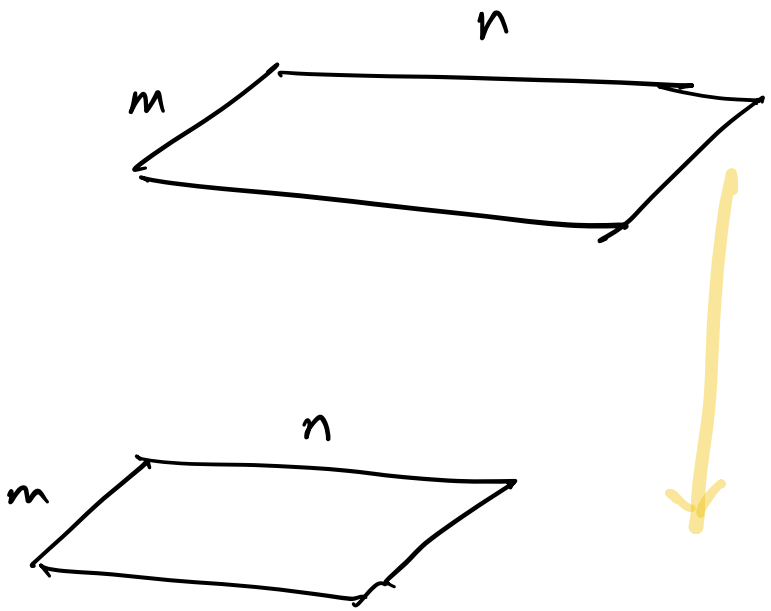
$\frac{\partial \underline{y}}{\partial \underline{\underline{W}}} \in \mathbb{R}^{m \times n \times m} \longrightarrow$ tensor

$$\underline{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}$$

$$\frac{\partial y_1}{\partial \underline{\underline{W}}}$$

$$\vdots$$

$$\frac{\partial y_m}{\partial \underline{\underline{W}}}$$



tensor $(m \times n) \times m$